

Algorithms

Lecture 5

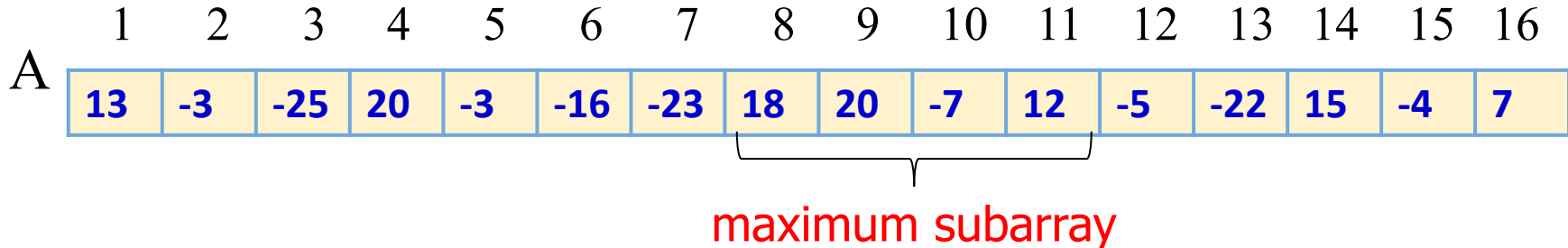
Max Sum Subarray (Divide and Conquer)

Lecturer

Department of Computer Science and Engineering
BRAC University

Maximum Subarray Problem

- *Input:* an array $A[1..n]$ of n numbers
 - Assume that some of the numbers are **negative**, because this problem is trivial when all numbers are nonnegative
- *Output:* a nonempty subarray $A[i..j]$ having the largest sum $S[i, j] = a_i + a_{i+1} + \dots + a_j$

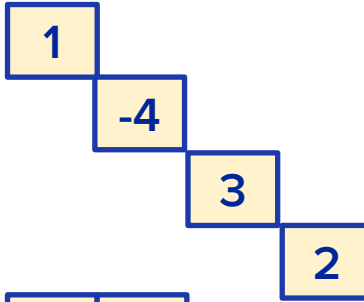


Maximum Subarray Problem

Target array :



All the sub arrays:

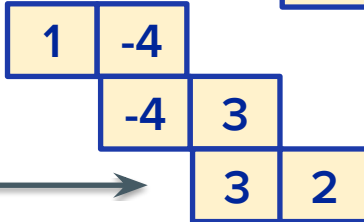


1

-4

3

2



-3

-1

5

Max! →



0

1



2

What is a maximum subarray?

Ans: The subarray with the largest sum

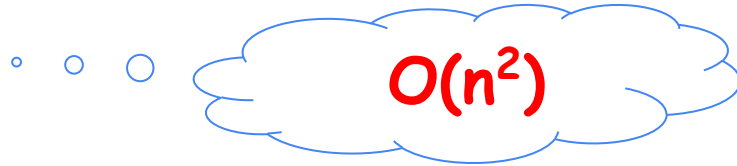
What is the brute-force time?

Brute-Force Algorithm

All possible contiguous subarrays

- $A[1..1], A[1..2], A[1..3], \dots, A[1..(n-1)], A[1..n]$
- $A[2..2], A[2..3], \dots, A[2..(n-1)], A[2..n]$
- ...
- $A[(n-1)..(n-1)], A[(n-1)..n]$
- $A[n..n]$

How many of them in total?



Algorithm: For each subarray, compute the sum.

Find the subarray that has the maximum sum.

Brute-Force Algorithm

Example:	2	-6	-1	3	-1	2	-2
sum from A[1]:	2	-4	-5	-2	-3	-1	-3
sum from A[2]:		-6	-7	-4	-5	-3	-5
sum from A[3]:			-1	2	1	3	1
sum from A[4]:				3	2	4	2
sum from A[5]:					-1	1	-1
sum from A[6]:						2	0
sum from A[7]:							-2

Brute-Force Algorithm

Outer loop: index variable i to indicate start of subarray, for $1 \leq i \leq n$, i.e., $A[1]$, $A[2]$, ..., $A[n]$

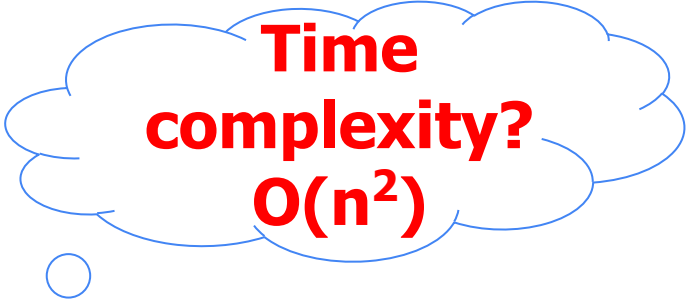
- for $i = 1$ to n do ...

Inner loop: for each start index i , we need to go through $A[i..i]$, $A[i..(i+1)]$, ..., $A[i..n]$

- use an index j for $i \leq j \leq n$, i.e., consider $A[i..j]$
- for $j = i$ to n do ...

Brute-Force Algorithm

```
max = -∞  
for i = 1 to n do  
begin  
    sum = 0  
    for j = i to n do  
begin  
    sum = sum + A[j]  
    if sum > max  
    then max = sum  
end  
end  
end
```

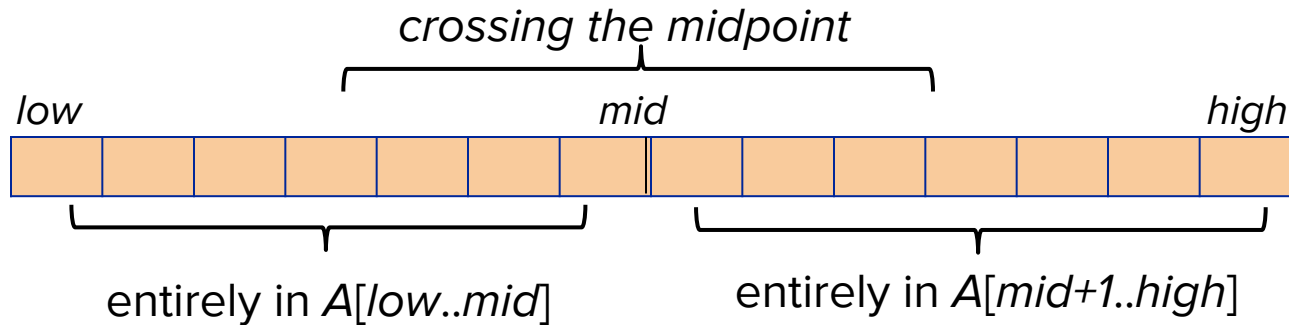


**Time
complexity?
 $O(n^2)$**

Divide-and-Conquer Algorithm

Possible locations of a maximum subarray $A[i..j]$ of $A[low..high]$, where $mid = \lfloor (low + high)/2 \rfloor$

- entirely in $A[low..mid]$ ($low \leq i \leq j \leq mid$)
- entirely in $A[mid+1..high]$ ($mid < i \leq j \leq high$)
- crossing the midpoint ($low \leq i \leq mid < j \leq high$)



Possible locations of subarrays of $A[low..high]$

Divide-and-Conquer Algorithm

FIND-MAX-CROSSING-SUBARRAY (A , low , mid , $high$)

$left-sum = -\infty$ // Find a maximum subarray of the form $A[i..mid]$

$sum = 0$

for $i = mid$ **downto** low

$sum = sum + A[i]$

if $sum > left-sum$

$left-sum = sum$

$max-left = i$

$right-sum = -\infty$ // Find a maximum subarray of the form $A[mid + 1 .. j]$

$sum = 0$

for $j = mid + 1$ **to** $high$

$sum = sum + A[j]$

if $sum > right-sum$

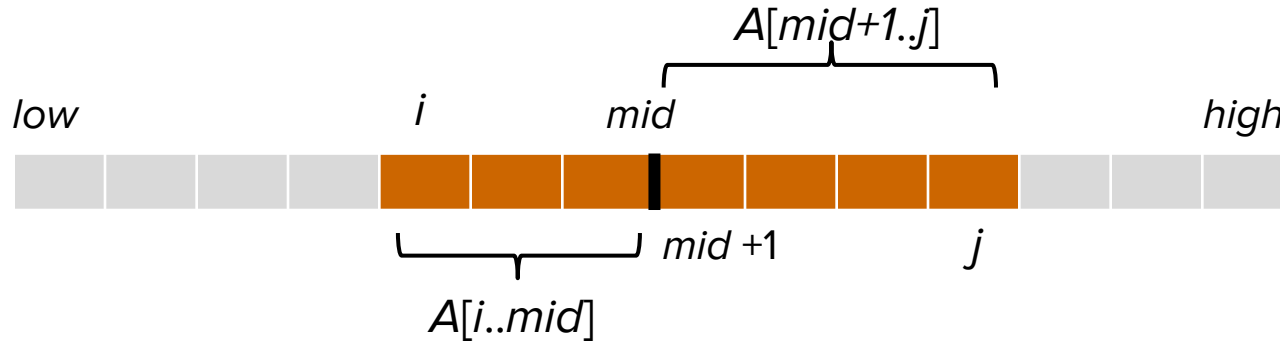
$right-sum = sum$

$max-right = j$

// Return the indices and the sum of the two subarrays

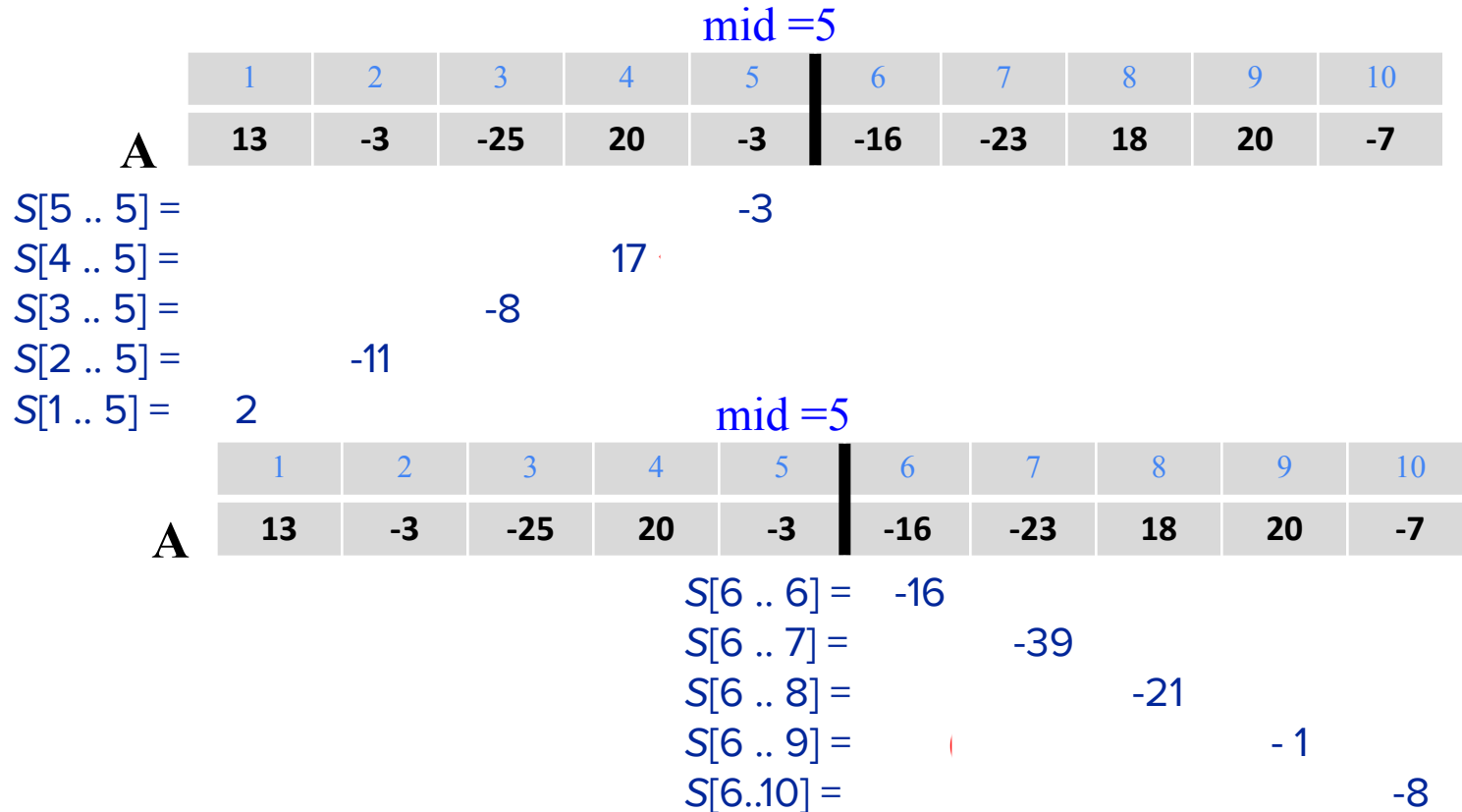
return ($max-left, max-right, left-sum + right-sum$)

Divide-and-Conquer Algorithm



$A[i..j]$ comprises two subarrays $A[i..mid]$ and $A[mid+1..j]$

Divide-and-Conquer Algorithm



$\Rightarrow$ maximum subarray crossing mid is $S[4 \dots 9] = 16$

Divide-and-Conquer Algorithm

FIND-MAXIMUM-SUBARRAY (*A*, *low*, *high*)

if *high* == *low*

return (*low*, *high*, *A*[*low*]) // base case: only one element

else *mid* = $\lfloor low + high / 2 \rfloor$

 (*left-low*, *left-high*, *left-sum*) =

FIND-MAXIMUM-SUBARRAY(*A*, *low*, *mid*)

 (*right-low*, *right-high*, *right-sum*) =

FIND-MAXIMUM-SUBARRAY(*A*, *mid* + 1, *high*)

 (*cross-low*, *cross-high*, *cross-sum*) =

FIND-MAX-CROSSING-SUBARRAY(*A*, *low*, *mid*, *high*)

if *left-sum* $\geq$ *right-sum* **and** *left-sum* $\geq$ *cross-sum*

return (*left-low*, *left-high*, *left-sum*)

elseif *right-sum* $\geq$ *left-sum* **and** *right-sum* $\geq$ *cross-sum*

return (*right-low*, *right-high*, *right-sum*)

else return (*cross-low*, *cross-high*, *cross-sum*)

Initial call: FIND-MAXIMUM-SUBARRAY (A, 1, n)

Divide-and-Conquer Algorithm

Analyzing time complexity

FIND-MAX-CROSSING-SUBARRAY : $\Theta(n)$, where $n = high - low + 1$

FIND-MAXIMUM-SUBARRAY

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

$$\begin{aligned} T(n) &= 2T(n/2) + \Theta(n) \\ &= \Theta(n \lg n) \quad (\text{similar to merge-sort}) \end{aligned}$$

Practice

-2	1	-3	4	-1	2	1	-5	4
----	---	----	---	----	---	---	----	---

Can we do better?

*the array from left to right and for each element, find the maximum sum among **all subarrays ending at that element**. The result will be the maximum of all these values.*

But, the main issue is how to calculate maximum sum among all the subarrays ending at an element in $O(1)$ time?

Can we do better?

*To calculate the maximum sum of subarray ending at current element, say **maxEnding**, we can use the maximum sum ending at the previous element. So for any element, we have two choices:*

- **Choice 1:** *Extend the maximum sum subarray ending at the previous element by adding the current element to it. If the maximum subarray sum ending at the previous index is **positive**, then it is always better to extend the subarray.*

Can we do better?

- **Choice 2:** *Start a new subarray starting from the current element. If the maximum subarray sum ending at the previous index is **negative**, it is always better to start a new subarray from the current element.*

Can we do better?

*This means that **maxEnding at index i** = **$\max(\text{maxEnding at index } (i - 1) + \text{arr}[i], \text{arr}[i])$** and the **maximum** value of **maxEnding** at any index will be our answer.*

Can we do better?

```
def maxSubarraySum(arr):  
    ans = arr[0]  
    maxEndingHere = arr[0]  
  
    for i in range(1, len(arr)):  
        # Find the maximum sum ending at index i  
        # by either extending the maximum sum subarray ending at index i - 1  
        # (when maxEndingHere positive) or  
        # by starting a new subarray from index i (when maxEndingHere negative)  
        maxEndingHere = max(maxEndingHere + arr[i], arr[i])  
  
        # Update res if maximum subarray sum ending at index i > res  
        ans = max(ans, maxEndingHere)  
  
    return ans
```

```
arr = [2, 3, -8, 7, -1, 2, 3]  
print(maxSubarraySum(arr))
```

Practice

-2	1	-3	4	-1	2	1	-5	4
----	---	----	---	----	---	---	----	---

Resources

- [CLRS Book:](#)
 - Chapter 4.1 - The Maximum Subarray Problem
- [Kadane's Algorithm](#)
- [Kadane's Algo visualizer](#)